Tugba Ates

MSc Computer & Information Science, University of Konstanz | KNIME Analytics Platform Portfolio — Detailed Methodology

This document walks through exactly how each of the two KNIME workflows below was built, node by node, along with the actual workflow diagrams and result charts exported directly from KNIME Analytics Platform. Both workflows are public on my KNIME Hub profile: hub.knime.com/tugbaates.

1 Spotify Tracks Analysis

Goal: clean a large real-world music dataset, understand what drives a song's popularity, and build a model that predicts it from audio characteristics alone.

114,000	113,999	$R^2 = 0.54$
Tracks loaded	Rows after cleaning	Model accuracy

METHODOLOGY

1. Data ingestion

A **CSV Reader** node loads a public 114,000-row Spotify tracks dataset (20 columns: track metadata, genre, popularity, and 10 numeric audio features such as danceability, energy, loudness, and tempo).

2. Cleaning

A **Missing Value** node removes the small number of rows with incomplete core metadata (missing artist/album/track name), using a "Remove Row" policy on all String columns. 113,999 of 114,000 rows survive — the dataset was already largely clean.

3. Genre exploration

A **GroupBy** node computes the mean popularity per genre (125 genres), a **Top k Row Filter** isolates the 10 highest-scoring genres, and the result is rendered with an interactive **Bar Chart** node (see result below).

4. Feature relationships

A **Linear Correlation** node computes the pairwise Pearson correlation across the 10 numeric audio features, producing a full correlation matrix used to sanity-check which features carry independent signal before modeling.

5. Train / test split

A **Table Partitioner** splits the cleaned data 80/20 into training and test sets using random sampling with a fixed seed, so results are reproducible.

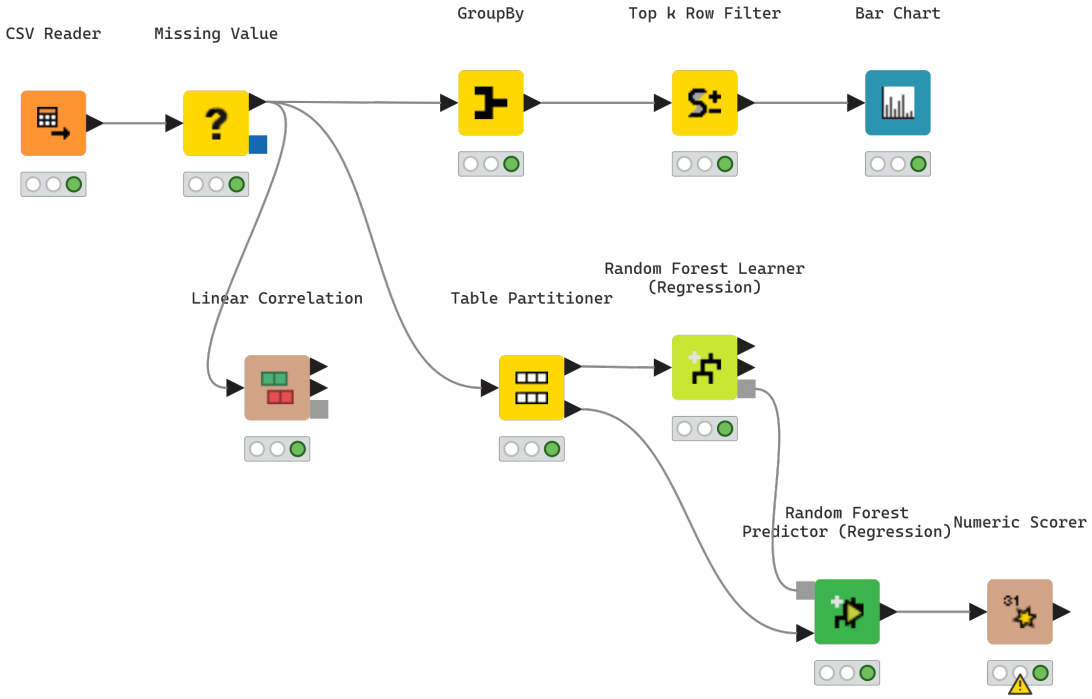
6. Model training

A **Random Forest Learner (Regression)** with 100 trees is trained on the 80% training split, using the 10 audio features as predictors and *popularity* as the numeric target — deliberately excluding popularity itself from the feature set to avoid leakage.

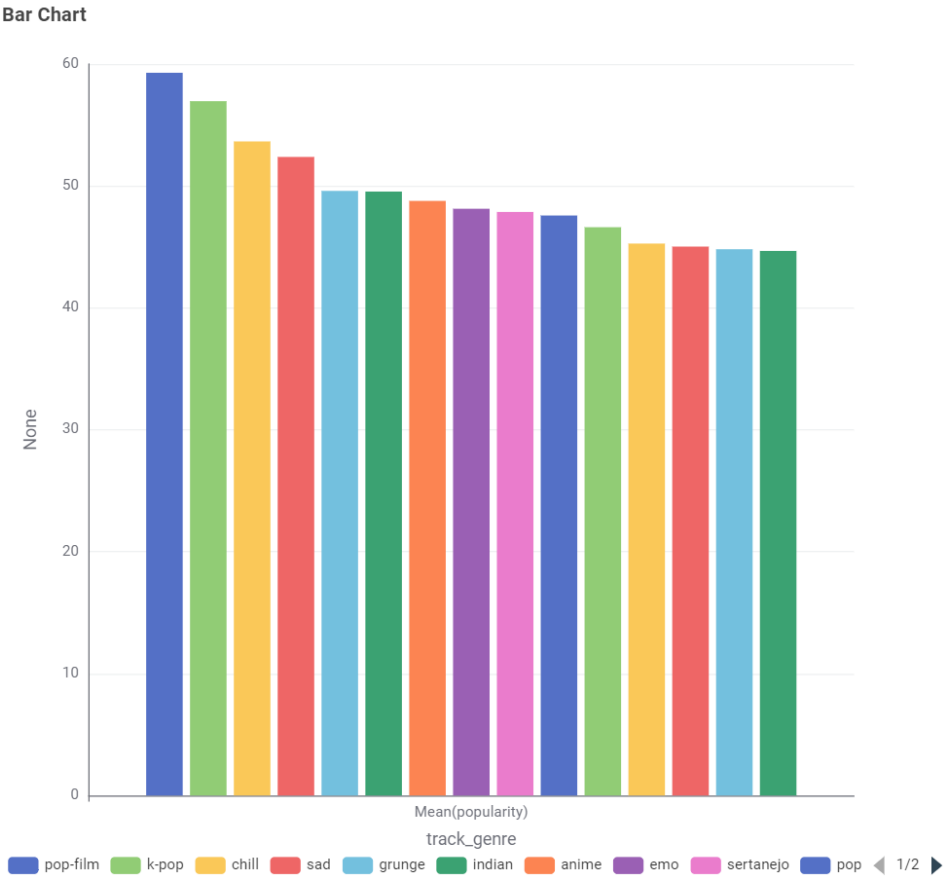
7. Prediction & evaluation

A **Random Forest Predictor (Regression)** applies the trained model to the held-out 20% test set, and a **Numeric Scorer** compares predicted vs. actual popularity, reporting **$R^2 = 0.539$** and a mean absolute error of ~10 points (0–100 scale).

ACTUAL WORKFLOW (exported from KNIME)



RESULT: TOP GENRES BY AVERAGE POPULARITY



Screenshot taken directly from the interactive Bar Chart view in KNIME. Pop-film, k-pop, and chill are the most consistently popular genres. The correlation step also showed a strong positive relationship between *energy* and *loudness*, and a strong negative one between *energy* and *acousticness* — both intuitive and useful signals that likely helped the Random Forest model.

2 KNIME Community Forum Trend Analysis

Goal: pull live data directly from KNIME's own public community forum and find out what the community is currently most active around — both by category and by the actual words used in discussion titles.

Live API	90 → 82	341
forum.knime.com	Topics processed / enriched	Unique keywords found

METHODOLOGY

1. Live data collection

Three **GET Request** nodes call KNIME forum's public Discourse API (`forum.knime.com/latest.json?page=0,1,2`, no authentication required), retrieving 30 of the latest topics per page (90 total). A **Concatenate** node combines the three raw JSON responses into one table.

2. JSON parsing

A **JSON Path** node applies the expression `$.topic_list.topics[*]` to pull the array of topic objects out of each page's JSON. An **Ungroup** node explodes that list so each topic becomes its own row, and a **JSON to Table** node flattens each topic's nested fields (title, views, like_count, reply_count, category_id, created_at, tags) into real columns, which a **Column Filter** then trims down to the ones actually needed.

3. Enrichment

The API only returns numeric category IDs, so a small lookup table (built with a **Table Creator** node, mapping all 15 known category IDs to their real names, e.g. 20 → "KNIME Analytics Platform") is joined onto the data with a **Joiner** node. A **Math Formula** node then computes a composite engagement score per topic: `views + 5×likes + 3×replies`, weighting active engagement above passive views.

4. Category activity analysis

A **GroupBy** node counts topics per category, rendered with a **Bar Chart** node (see result below).

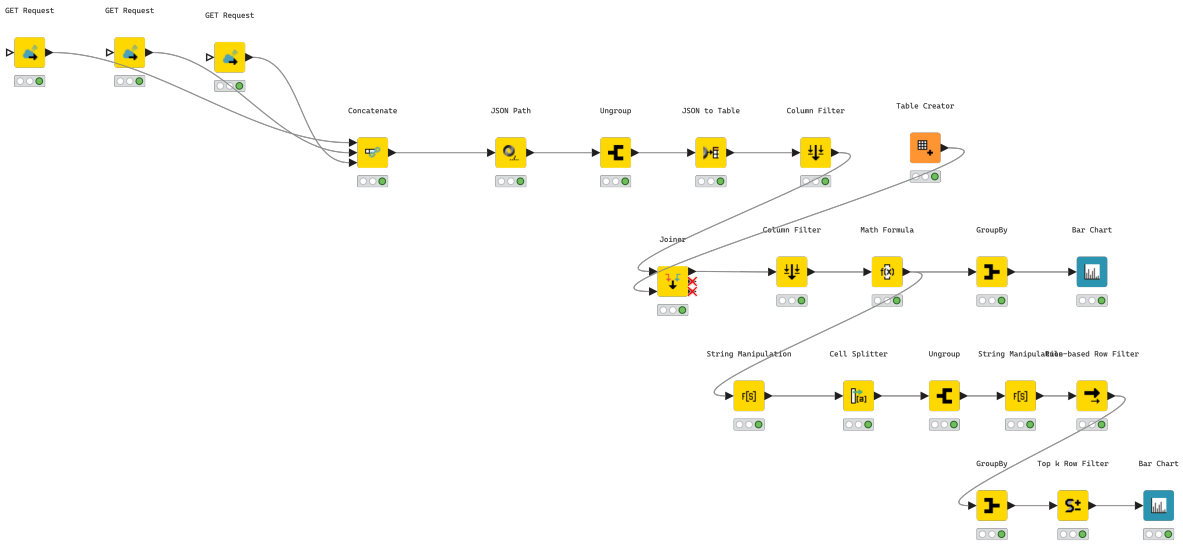
5. Text preparation

Titles are lowercased (**String Manipulation**), split into individual words (**Cell Splitter** + **Ungroup**), and stripped of punctuation with a regex replace (a second **String Manipulation** step) — this step also surfaced a small real data-quality quirk: a Unicode character in one post's stylized "KNIME" was stripped to "knme" and had to be added to the stop-word list by hand.

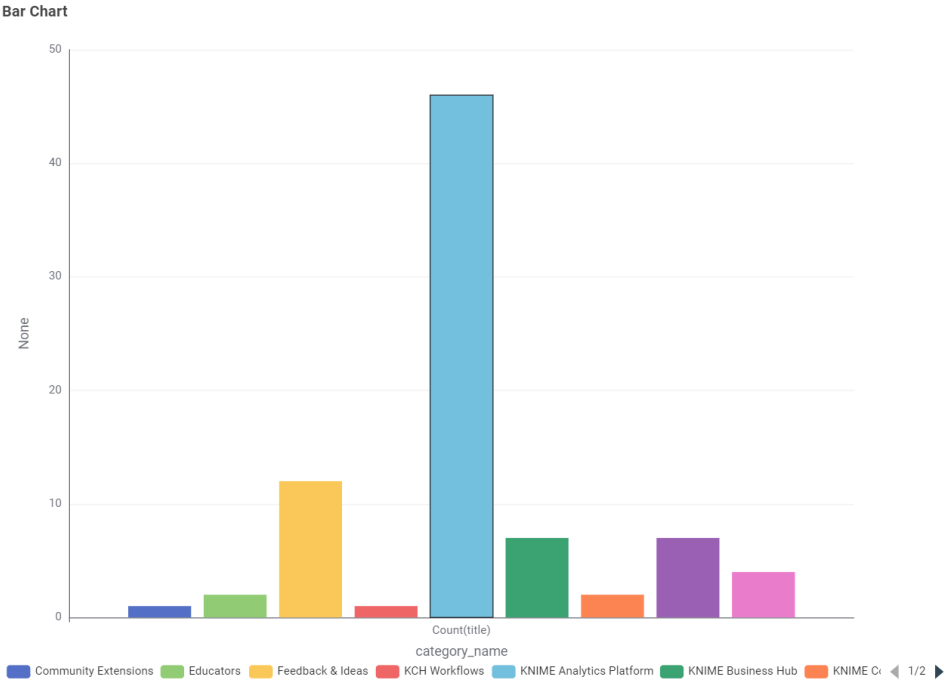
6. Keyword frequency

A **Rule-based Row Filter** removes common stop-words and generic platform terms (the, and, knime, node, workflow, etc.) so the remaining words are genuinely distinctive. A second **GroupBy** counts frequency per word, and a **Top k Row Filter** keeps the 20 most frequent, rendered with a final **Bar Chart** node.

ACTUAL WORKFLOW (exported from KNIME)

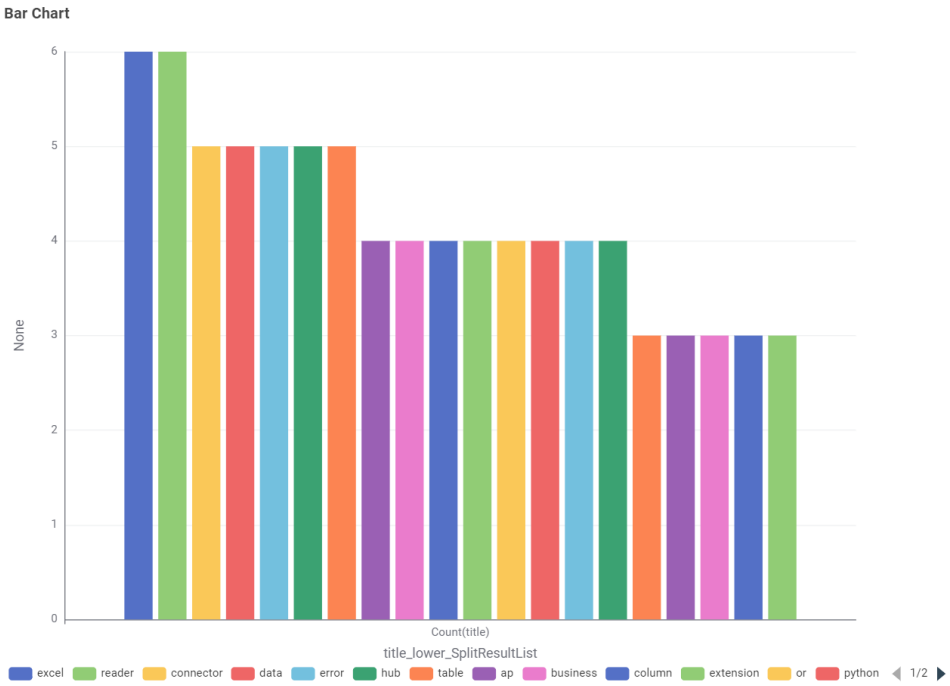


RESULT: MOST ACTIVE CATEGORIES



The **KNIME Analytics Platform** category dominates recent activity by a wide margin, followed by Feedback & Ideas.

RESULT: MOST FREQUENT KEYWORDS IN RECENT TITLES



Keywords like *excel, reader, connector, data, error, hub, business,* and *python* dominate — suggesting file I/O, connector configuration, and Python integration are where the community is currently most active.